



Android Penetration Testing

MOBSF

FRAMEWORK

Contents

Abstract	3
Installation	4
Exploring MobSF	6
Landing Page	6
Signer Certificate Analysis.....	8
Application Permissions	8
Browsable Activities & Network Security Analysis.....	9
Manifest Analysis	11
Code Analysis	12
Related to Malware Analysis	13
Hardcoded secrets.....	17
Activity Components Present.....	17
Dynamic Analyzer.....	20
Logcat Stream	24
API Monitor.....	25
Downloading Reports	26

Abstract

MobSF is an open-source tool developed by Ajin Abraham that is used for automated analysis of an APK. This is a collection of tools that run under one interface, perform their own individual tasks (like Jadx, apktool etc) and display their results under a common interface. These reports can be downloaded in a PDF format too and give out detailed analysis with necessary screenshots as well. You can download MobSF [here](#). In this publication, we'll be walking through the installation phase in Ubuntu OS and guiding you through various options that this tool has to offer.

Installation

To install MobSF, create a directory and follow the commands:

```
git clone https://github.com/MobSF/Mobile-Security-Framework-MobSF.git
cd Mobile-Security-Framework-MobSF
```

```
root@hex:/home/hex/android-toolkit# git clone https://github.com/MobSF/Mobile-Security-Framework-MobSF.git
Cloning into 'Mobile-Security-Framework-MobSF'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 16859 (delta 0), reused 1 (delta 0), pack-reused 16856
Receiving objects: 100% (16859/16859), 1.09 GiB | 7.64 MiB/s, done.
Resolving deltas: 100% (8022/8022), done.
root@hex:/home/hex/android-toolkit# cd Mobile-Security-Framework-MobSF/
root@hex:/home/hex/android-toolkit/Mobile-Security-Framework-MobSF#
```

We need to install dependencies before we are able to run:

```
apt-get install python3-venv
pip3 install -r requirements.txt
```

```
root@hex:/home/hex/android-toolkit/Mobile-Security-Framework-MobSF# apt-get install python3-venv
Reading package lists... Done
Building dependency tree
Reading state information... Done
python3-venv is already the newest version (3.8.2-0ubuntu2).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
root@hex:/home/hex/android-toolkit/Mobile-Security-Framework-MobSF# pip3 install -r requirements.txt
Ignoring waitress: markers 'platform_system == "Windows"' don't match your environment
Requirement already satisfied: Django>=3.1.5 in /usr/local/lib/python3.8/dist-packages (from -r requirements.txt (line 1)) (3.1.7)
Requirement already satisfied: lxml>=4.6.2 in /usr/local/lib/python3.8/dist-packages (from -r requirements.txt (line 2)) (4.6.2)
```

Once done, we can run the setup file to install MobSF and all the components automatically

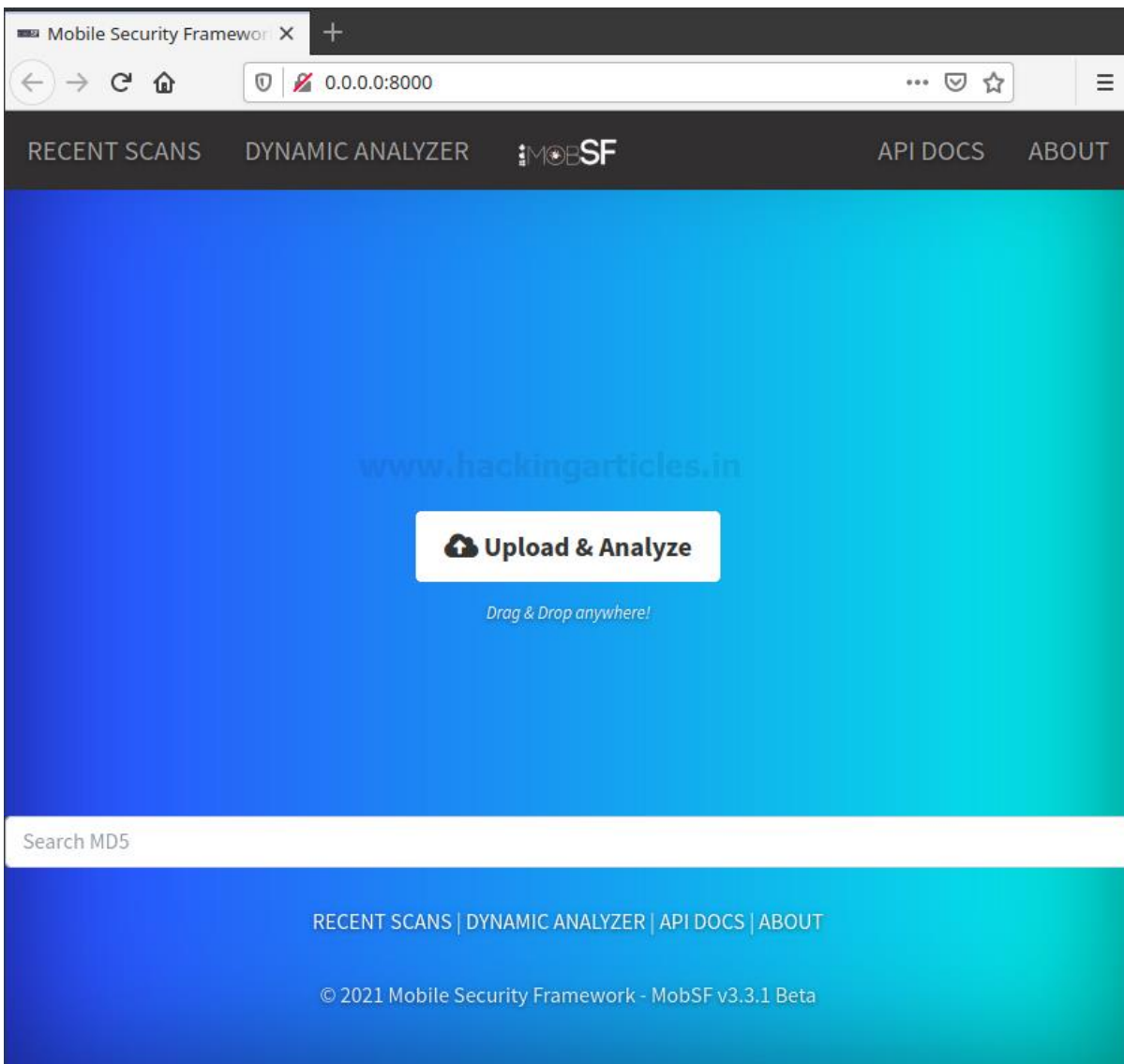
```
./setup.sh
```

```
root@hex:/home/hex/android-toolkit/Mobile-Security-Framework-MobSF# ./setup.sh
[INSTALL] Found Python 3.8.5
pip 21.0.1 from /root/.local/lib/python3.8/site-packages/pip (python 3.8)
[INSTALL] Found pip
Requirement already satisfied: pip in /root/.local/lib/python3.8/site-packages (21.0.1)
[INSTALL] Using python virtualenv
```

Now, to run MobSF we execute the **run.sh** file. As one could interpret from the screenshot below that MobSF would be running on a local server on port 8000.

```
root@hex:/home/hex/android-toolkit/Mobile-Security-Framework-MobSF# ./run.sh
[2021-02-22 21:15:01 +0530] [15422] [INFO] Starting gunicorn 20.0.4
[2021-02-22 21:15:01 +0530] [15422] [INFO] Listening at: http://0.0.0.0:8000 (15422)
[2021-02-22 21:15:01 +0530] [15422] [INFO] Using worker: threads
[2021-02-22 21:15:01 +0530] [15424] [INFO] Booting worker with pid: 15424
```

Now let's open the link in the browser and see if MobSF was installed properly or not.



Exploring MobSF

Landing Page

Now that the MobSF is up and running, we can drag a dummy APK (in this case, I'll take InjuredAndroid by Kyle Benac ([here](#)) into the MobSF interface and see what happens. After waiting for a couple of minutes we could see that static analysis of the APK is done. Now here on the landing page, we can see that a severity score is given. The higher this score the more secure app is. Next, hashes, filename and size of the APK are also given. In the third column in the first row, we can also see the package name, main activity, min SDK version and the application version as well. The description of the application is also given.

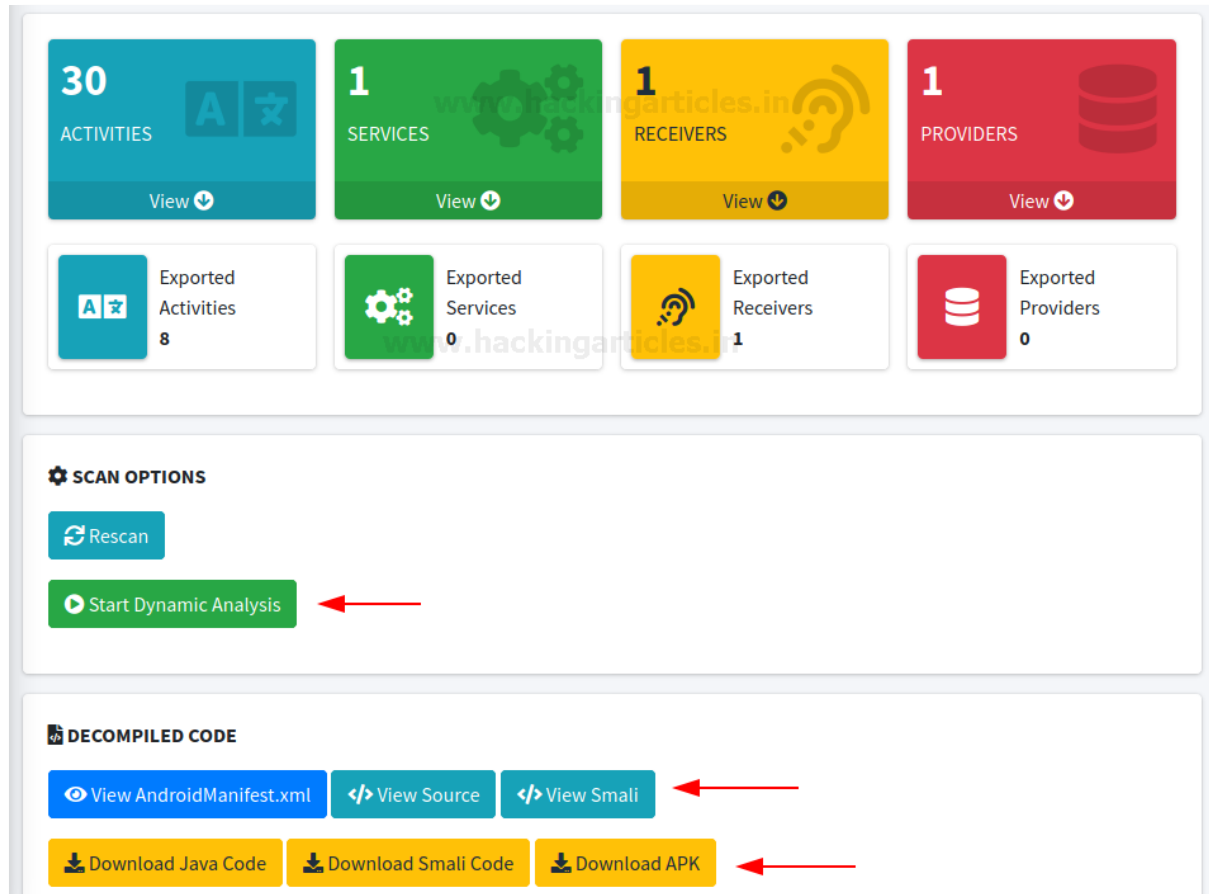
The screenshot displays the MobSF Static Analysis interface. The browser address bar shows the URL: `0.0.0.0:8000/static_analyzer/?name=InjuredAndroid-1.0.10-release.apk&ch`. The interface is divided into several sections:

- APP SCORES:** Shows an Android icon, Average CVSS 7.0, Security Score 70/100, and Trackers Detection 0/343.
- FILE INFORMATION:** Lists File Name (InjuredAndroid-1.0.10-release.apk), Size (17.49MB), MD5 (c7c91424ab41179ead7186cf586c8a3d), SHA1 (47d86e733821cec0cb5dae8eab68edddedf0de10), and SHA256 (5e111b2bdcd9c9550737ec468bd30ddfb6e4af40bf61a573c775a27d61eae02).
- APP INFORMATION:** Lists App Name (InjuredAndroid), Package Name (b3nac.injuredandroid), Main Activity (b3nac.injuredandroid.MainActivity), Target SDK (29), Min SDK (21), Max SDK, Android Version Name (1.0.9), and Android Version Code (17).
- PLAYSTORE INFORMATION:** Lists Title (InjuredAndroid), Score (0.0), Installs (100+), Price (0), Android Version Support (5.0 and up), Category (Education), Play Store URL (b3nac.injuredandroid), Developer (B3nac), Developer ID (B3nac), Developer Address (None), Developer Website (https://b3nac.com/), Developer Email (b3nac.sec@gmail.com), Release Date (Jul 20, 2020), Privacy Policy (Privacy link), and Description (Setup for a physical device, 1. Download the latest release from Google Play., Setup for an Android Emulator using Android Studio, 1. Pull the apk from a physical device after installing from Google Play with adb pull.).

After scrolling down a little bit, here's what all we can see:

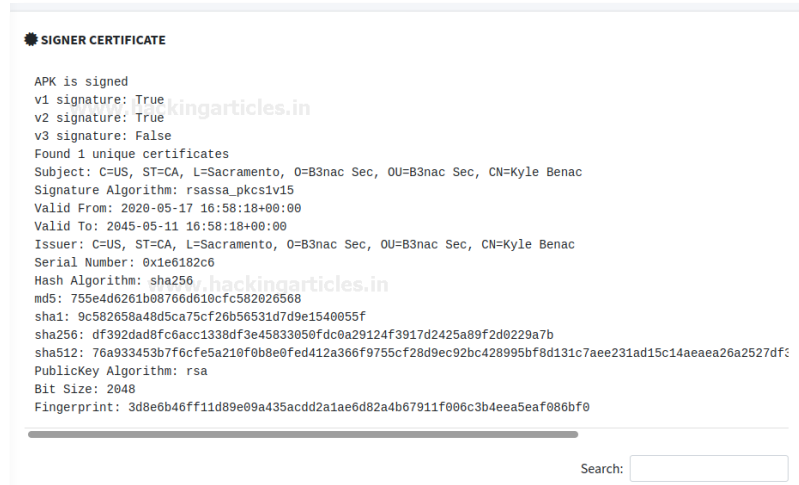
In small cards, we see different application components

Dynamic analysis option that will help MobSF conduct run time analyses Option to view decompiled code. This is the code that is generated by apktool. Generally, the resources file would also be decoded. It is also possible to view smali code. It makes it easier to segregate and view source code in separate java classes using this.



Signer Certificate Analysis

In the certificate column, we can see the signer certificate where one can find important information about the developer, country, state, type of algo, bit size etc.



Application Permissions

Further, we can see all the permissions an application has. There are various permissions that are categorized as dangerous or normal. It is important from a security analyst's point of view to understand which permissions can lead to further damage. For example, if an application has access to external media and stores critical information on the external media it could prove to be dangerous since the files stored on external media are globally readable and writable.

APPLICATION PERMISSIONS

Search:

PERMISSION	STATUS	INFO	DESCRIPTION
android.permission.ACCESS_NETWORK_STATE	normal	view network status	Allows an application to view the status of all networks.
android.permission.INTERNET	normal	full Internet access	Allows an application to create network sockets.
android.permission.READ_EXTERNAL_STORAGE	dangerous	read external storage contents	Allows an application to read from external storage.
android.permission.READ_PHONE_STATE	dangerous	read phone state and identity	Allows the application to access the phone features of the device. An application with this permission can determine the phone number and serial number of this phone, whether a call is active, the number that call is connected to and so on.
android.permission.WRITE_EXTERNAL_STORAGE	dangerous	read/modify/delete external storage contents	Allows an application to write to external storage.

Showing 1 to 5 of 5 entries

Previous 1 Next

Browsable Activities & Network Security Analysis

Next, in the browsable activities section, we can see all the activities that have implemented a deep link schema. Please refer to the article [here](#) to understand all about deep links, its implementation as well as exploitation.

In the network security section, one can find some details about network security issues related to the application. These issues can lead to critical attacks like MiTM sometimes. For example, in the screenshot below, one can find that the application isn't using the SSL pinning mechanism implemented.

BROWSABLE ACTIVITIES

Search:

ACTIVITY	INTENT
b3nac.injuredandroid.CSPBypassActivity	Schemes: http://, https://, Hosts: b3nac.com, Path Patterns: /*,
b3nac.injuredandroid.DeepLinkActivity	Schemes: flag11://, https://,
b3nac.injuredandroid.RCEActivity	Schemes: flag13://, Hosts: rce,

Showing 1 to 3 of 3 entries

Previous **1** Next

NETWORK SECURITY

Search:

NO	SCOPE	SEVERITY	DESCRIPTION
1	*	high	Base config is insecurely configured to permit clear text traffic to all domains.

Showing 1 to 1 of 1 entries

Previous **1** Next

Manifest Analysis

In the next section, MobSF has analysed the manifest file. One can find many folds of information from the android manifest file like which activities are exported, if the app debuggable or not, data schemas etc. For reference look at the screenshot below.

MANIFEST ANALYSIS			
		Search:	
NO ↑↓	ISSUE ↑↓	SEVERITY ↑↓	DESCRIPTION ↑↓
1	App has a Network Security Configuration [android:networkSecurityConfig=@xml/network_security_config]	Info	The Network Security Configuration feature lets apps customize their network security settings in a safe, declarative configuration file without modifying app code. These settings can be configured for specific domains and for a specific app.
2	Application Data can be Backed up [android:allowBackup] flag is missing.	medium	The flag [android:allowBackup] should be set to false. By default it is set to true and allows anyone to backup your application data via adb. It allows users who have enabled USB debugging to copy application data off of the device.
3	Activity (b3nac.injuredandroid.CSPBypassActivity) is not Protected. An intent-filter exists.	high	An Activity is found to be shared with other apps on the device

Code Analysis

One of the most interesting features of the MobSF tool is the code analysis section. In this section, we can see that MobSF has analysed and compared some behaviour of the application based on industry security standard practices like OWASP MSTG and mapped the vulnerabilities with OWASP Top 10. It is interesting to see CWE mentioned and CVSS score being assigned here which might help various analyst scenarios and help the creation of reports way easier.

</> CODE ANALYSIS				
Search:				
NO	ISSUE	SEVERITY	STANDARDS	FILES
1	App uses SQLite Database and execute raw SQL query. Untrusted user input in raw SQL queries can cause SQL Injection. Also sensitive information should be encrypted and written to the database.	high	CVSS V2: 5.9 (medium) CWE: CWE-89 Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') OWASP Top 10: M7: Client Code Quality	b3nac/injuredandroid/f.java
2	The App logs information. Sensitive information should never be logged.	info	CVSS V2: 7.5 (high) CWE: CWE-532 Insertion of Sensitive Information into Log File OWASP MASVS: MSTG-STORAGE-3	b/c/c/c.java b3nac/injuredandroid/FlagFiveReceiver.java b3nac/injuredandroid/k.java a/i/b/c.java b/c/a/a/b/d.java b3nac/injuredandroid/AssemblyActivity.java b/c/a/b/y/b.java b/c/a/a/d/c/q1.java a/g/e/g.java io/flutter/view/c.java a/g/j/b.java b/c/a/b/n/a.java a/g/d/d/b.java c/a/a.java a/g/l/b.java a/a/n/g.java

Related to Malware Analysis

MobSF also hosts a section where an APKiD analysis is given. APKiD is an open-source tool that is very helpful to identify various packers, compilers, obfuscators etc in android files. It is analogous to PEiD in APK. Here one can see that it has detected an anti-vm code in the APK.

FILE ANALYSIS
Search:

NO	ISSUE	FILES
No data available in table		

Showing 0 to 0 of 0 entriesPreviousNext

APKiD ANALYSIS
Search:

DEX	DETECTIONS						
classes.dex	Search: <table><thead><tr><th>FINDINGS</th><th>DETAILS</th></tr></thead><tbody><tr><td>Anti-VM Code</td><td>Build.MANUFACTURER check</td></tr><tr><td>Compiler</td><td>unknown (please file detection issue!)</td></tr></tbody></table> Showing 1 to 2 of 2 entriesPrevious1Next	FINDINGS	DETAILS	Anti-VM Code	Build.MANUFACTURER check	Compiler	unknown (please file detection issue!)
FINDINGS	DETAILS						
Anti-VM Code	Build.MANUFACTURER check						
Compiler	unknown (please file detection issue!)						

Showing 1 to 1 of 1 entriesPrevious1Next

Something related to malware analysis is the domain malware check feature. Here, MobSF is extracting all the URLs/IP addresses that are hard-coded or being used in the application and shows its malware status as well as uses ip2location to give out its geolocation as well.

DOMAIN MALWARE CHECK

Search:

DOMAIN	STATUS	GEOLOCATION
github.com	good	IP: 13.234.176.102 Country: India Region: Maharashtra City: Mumbai Latitude: 19.01441 Longitude: 72.847939 View: Google Map
injuredandroid.firebaseio.com	good	IP: 35.201.97.85 Country: United States of America Region: Missouri City: Kansas City Latitude: 39.099731 Longitude: -94.578568 View: Google Map
m.do.co	good	IP: 104.21.61.61 Country: United States of America Region: California City: San Francisco Latitude: 37.7757 Longitude: -122.395203 View: Google Map

A comprehensive strings analysis is also available. People who are aware of malware analysis know about strings in-depth but for those of you who don't, strings are ASCII and Unicode-printable sequences of characters embedded within a file. Extracting strings can give clues about the program functionality and indicators associated with a suspect binary. For example, if an APK shows something as an output so that stream would be called and hence shown in the strings. This is not the same as strings.xml file. Many times, a third party IP address with which APK is communicating gets visible here. This is essential from a malware analysis point of view.

A STRINGS

```
id-aes256-wrap
Icircumflexsmall
half4 color = half4(%s, %s, %s, %s);
"abc_searchview_description_query" : "ಪ್ರಶ್ನೆಯನ್ನು ಹುಡುಕಿ"
_input
"common_google_play_services_install_button" : "Инсталиране"
blend_dst_in
experimental
Private_Use_Area
float2 ab = mix(P[0], P[1], T);
Math_Alphanum
Dart_NewStringFromCString
Khojki
"abc_searchview_description_search" : "शोध"
Separator
UNABLE_TO_CREATE_NEW_SECTION
"abc_prepend_shortcut_label" : "Menú +"
"abc_capital_on" : "ಆನ್"
UNKNOWN_EXTENSION_NAME
../../third_party/libcxxabi/src/abort_message.cpp
AHZzsm
[%-8s : sp(%#x) fp(%#x) pc(%#x) %s%s ]
"abc_activitychooserview_choose_application" : "Escolher uma aplicação"
GrRenderTargetContext::drawTextureSet
glBindAttribLocation
"common_google_play_services_install_text" : "%1$s нема да се извршува без услугите на Google Play што ги нем
"abc_searchview_description_clear" : "क्वेरी साफ करा"
Show invisible frames in stack traces.
noexcept
"abc_menu_sym_shortcut_label" : "Sym+"
sInt64List.
ENTITIES
"status_bar_notification_info_overflow" : "+999"
null-safety
Surrogate
```

One can also find hardcoded emails in MobSF. This is all done using the decompiled source code. Often a pentester can find critical email IDs that were being used as a credential on a third party site, say, to access the database.

EMAILS

Search:

EMAIL	FILE
appro@openssl.org	lib/arm64-v8a/libflutter.so
b3nac.sec@gmail.com	b3nac/injuredandroid/ContactActivity.java
u0013android@android.com0 u0013android@android.com	b/c/a/a/b/v.java

Showing 1 to 3 of 3 entries

Previous1Next

Just like emails, URLs are often found hardcoded as well. One can find juicy URLs that are being used sometimes. Oftentimes analysts find malicious URLs being accessed as well or even a C&C server.

URLS

Search:

URL	FILE
http://localhost	b/c/a/a/d/c/a2.java
http://schemas.android.com/apk/res/android	a/g/d/d/g.java
http://www.w3.org/XML/1998/namespace data:application/dart data:application/dart; http://www.w3.org/2000/xmlns/ https://www.w3.org/Style/CSS/Test/Fonts/Ahem/).	lib/armeabi-v7a/libflutter.so
http://www.w3.org/XML/1998/namespace data:application/dart data:application/dart; http://www.w3.org/2000/xmlns/ https://www.w3.org/Style/CSS/Test/Fonts/Ahem/).	lib/arm64-v8a/libflutter.so
https://github.com/flutter/flutter/issues/2897).It	io/flutter/plugin/platform/i.java
https://injuredandroid.firebaseio.com	Android String Resource
https://m.do.co/c/9348bb7410b4	b3nac/injuredandroid/ContactActivity.java

Showing 1 to 7 of 7 entries

Previous1Next

Hardcoded secrets

Oftentimes developers have this habit of storing critical keys like AWS ID and credentials in strings.xml and use an object as a reference in java activity. But doing this doesn't help in any which way since strings.xml can be decoded easily.

POSSIBLE HARDCODED SECRETS

```
"AWS_ID" : "AKIAZ36DGKTUIOLDOBN6"  
"AWS_SECRET" : "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"  
"enter_password" : "Enter password"  
"firebase_database_url" : "https://injuredandroid.firebaseio.com"  
"flag_eight_aws" : "flag eight - aws"  
"flag_nine_firebase" : "flag nine - Firebase"  
"google_api_key" : "AlzaSyCUImEIOSvqAswLqFak75xhskkB6illd7A"  
"google_crash_reporting_api_key" : "AlzaSyCUImEIOSvqAswLqFak75xhskkB6illd7A"
```

Activity Components Present

A list of all the activities present can also be scrolled using MobSF. This gives an insight into the skeleton of the android APK. Also sometimes jadx replaces the real names of the class with some random letter if the developer has applied obfuscation, MobSF can associate its real name too (doesn't happen all the time or in cases of strong obfuscation).

ACTIVITIES

```
b3nac.injuredandroid.FlagSeventeenActivity
b3nac.injuredandroid.CSPBypassActivity
b3nac.injuredandroid.AssemblyActivity
io.flutter.embedding.android.FlutterActivity
b3nac.injuredandroid.RCEActivity
b3nac.injuredandroid.SettingsActivity
b3nac.injuredandroid.ExportedProtectedIntent
b3nac.injuredandroid.QXV0aA
b3nac.injuredandroid.FlagTwelveProtectedActivity
b3nac.injuredandroid.DeepLinkActivity
b3nac.injuredandroid.FlagTenUnicodeActivity
b3nac.injuredandroid.FlagOneLoginActivity
b3nac.injuredandroid.FlagNineFirebaseActivity
b3nac.injuredandroid.FlagEightLoginActivity
b3nac.injuredandroid.FlagSevenSqliteActivity
b3nac.injuredandroid.FlagsOverview
b3nac.injuredandroid.FlagSixLoginActivity
b3nac.injuredandroid.MainActivity
b3nac.injuredandroid.XSSTextActivity
b3nac.injuredandroid.DisplayPostXSS
b3nac.injuredandroid.FlagOneSuccess
b3nac.injuredandroid.b25IAActivity
b3nac.injuredandroid.FlagTwoActivity
b3nac.injuredandroid.FlagThreeActivity
b3nac.injuredandroid.FlagFourActivity
b3nac.injuredandroid.FlagFiveActivity
b3nac.injuredandroid.TestBroadcastReceiver
b3nac.injuredandroid.ContactActivity
com.google.firebase.auth.internal.FederatedSignInActivity
com.google.android.gms.common.api.GoogleApiActivity
```

Quite similarly an analyst can also traverse services, broadcast, providers and content receivers along with all the files present in the APK archive to create a map of all the resources present in the application.

SERVICES

com.google.firebase.components.ComponentDiscoveryService

RECEIVERS

b3nac.injuredandroid.FlagFiveReceiver

PROVIDERS

com.google.firebase.provider.FirebaseInitProvider

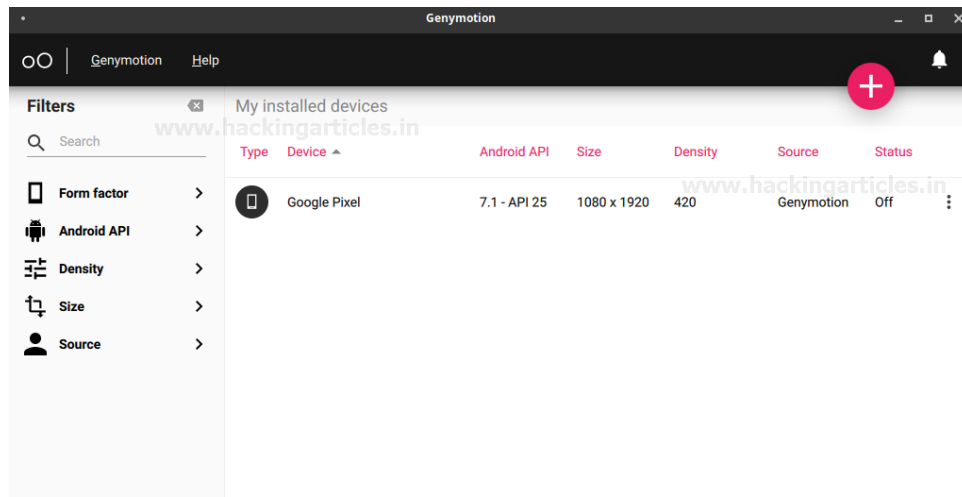
LIBRARIES

FILES

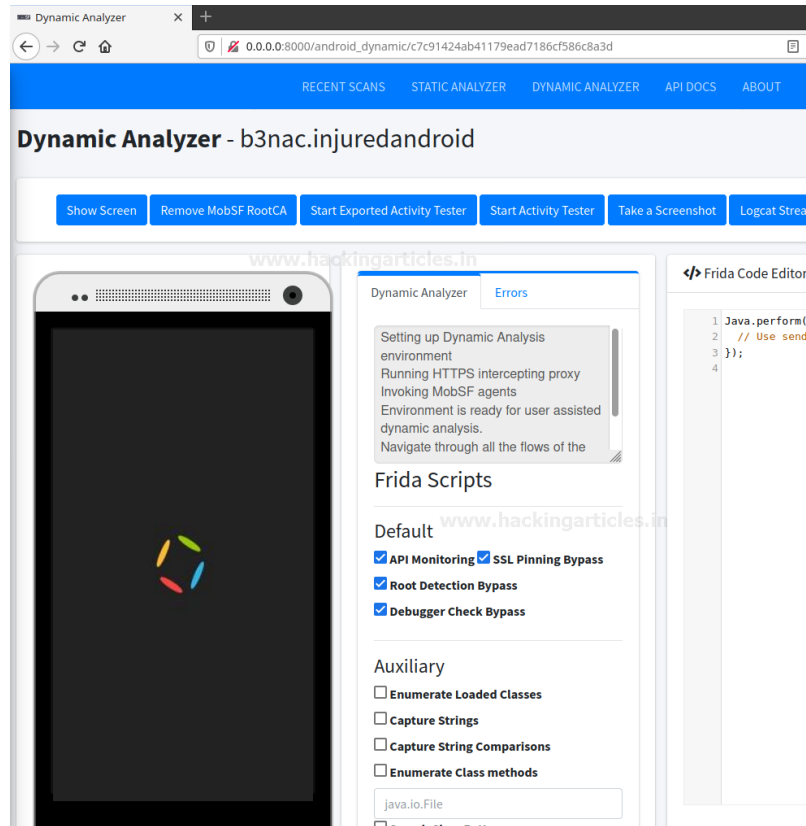
AndroidManifest.xml
META-INF/CERT.RSA
META-INF/CERT.SF
META-INF/MANIFEST.MF
META-INF/androidx.activity_activity.version
META-INF/androidx.appcompat_appcompat-resources.version
META-INF/androidx.appcompat_appcompat.version
META-INF/androidx.arch.core_core-runtime.version
META-INF/androidx.cardview_cardview.version
META-INF/androidx.coordinatorlayout_coordinatorlayout.version
META-INF/androidx.core_core-ktx.version
META-INF/androidx.core_core.version

Dynamic Analyzer

For dynamic analysis, we'd need to fire up android VM in genymotion first. Here I've created an android VM on version 7.1



When you press the dynamic analyser option present on the top navigation pane, MobSF will automatically attach itself to VM running if MobSF and genymotion are running on the same base machine. However, if MobSF is in another virtual machine, you might have to attach MobSF agent to genymotion's VM's remote IP and port. Once it is attached, we see the following screen.



Under the analyzer status bar we can see various default frida scripts available that would check various basic vulnerabilities like SSL Pinning bypass and Root detection checks. If you haven't read about frida, please do so by going [here](https://www.hackingarticles.in/frida/). There are other auxiliary scripts as well that lets an analyst enumerate various classes and also capture string comparisons in real time (again helpful for malware analysts point of view). Then simply click on start instrumentation and the selected scripts will be attached to the application automatically. Hence, if I have selected SSL Pinning bypass script and traffic is getting captured (visible in log or API monitor later) that would mean SSL Pinning has got bypassed.

Setting up Dynamic Analysis environment
Running HTTPS intercepting proxy
Invoking MobSF agents
Environment is ready for user assisted dynamic analysis.
Navigate through all the flows of the app manually.

www.hackingarticles.in

Frida Scripts

Default

☒ API Monitoring ☒ SSL Pinning Bypass ☒ Root Detection Bypass
☒ Debugger Check Bypass

Auxiliary

☒ Enumerate Loaded Classes ☒ Capture Strings
☒ Capture String Comparisons
☐ Enumerate Class methods
java.io.File
☐ Search Class Pattern
ssl.Trust*
☐ Trace Class Methods
java.net.Socket,java.io.File,java.lang.String

Start Instrumentation Frida Live Logs

Now further, to analyse activities for vulnerabilities one can see two buttons on the top for both exported and non exported activities

Dynamic Analyzer - b3nac.injuredandroid

www.hackingarticles.in

Show Screen Remove MobSF RootCA Start Exported Activity Tester Start Activity Tester Take a Screenshot



Dynamic Analyzer Errors

Setting up Dynamic Analysis environment
Running HTTPS intercepting proxy
Invoking MobSF agents
Environment is ready for user assisted dynamic analysis.
Navigate through all the flows of the app manually.
Instrumenting app with Frida
Successfully attached

Frida Scripts

Default

☒ API Monitoring ☒ SSL Pinning Bypass ☒ Root Detection Bypass
☒ Debugger Check Bypass

Auxiliary

☒ Enumerate Loaded Classes ☒ Capture Strings
☒ Capture String Comparisons
☐ Enumerate Class methods
java.io.File
☐ Search Class Pattern
ssl.Trust*

Similarly, if one doesn't have to make do with pre-configured Frida scripts, it is also possible that Frida script be pasted in the text box on the right. There is also a dropdown box that would load those scripts. You can also edit the same.

Frida Code Editor

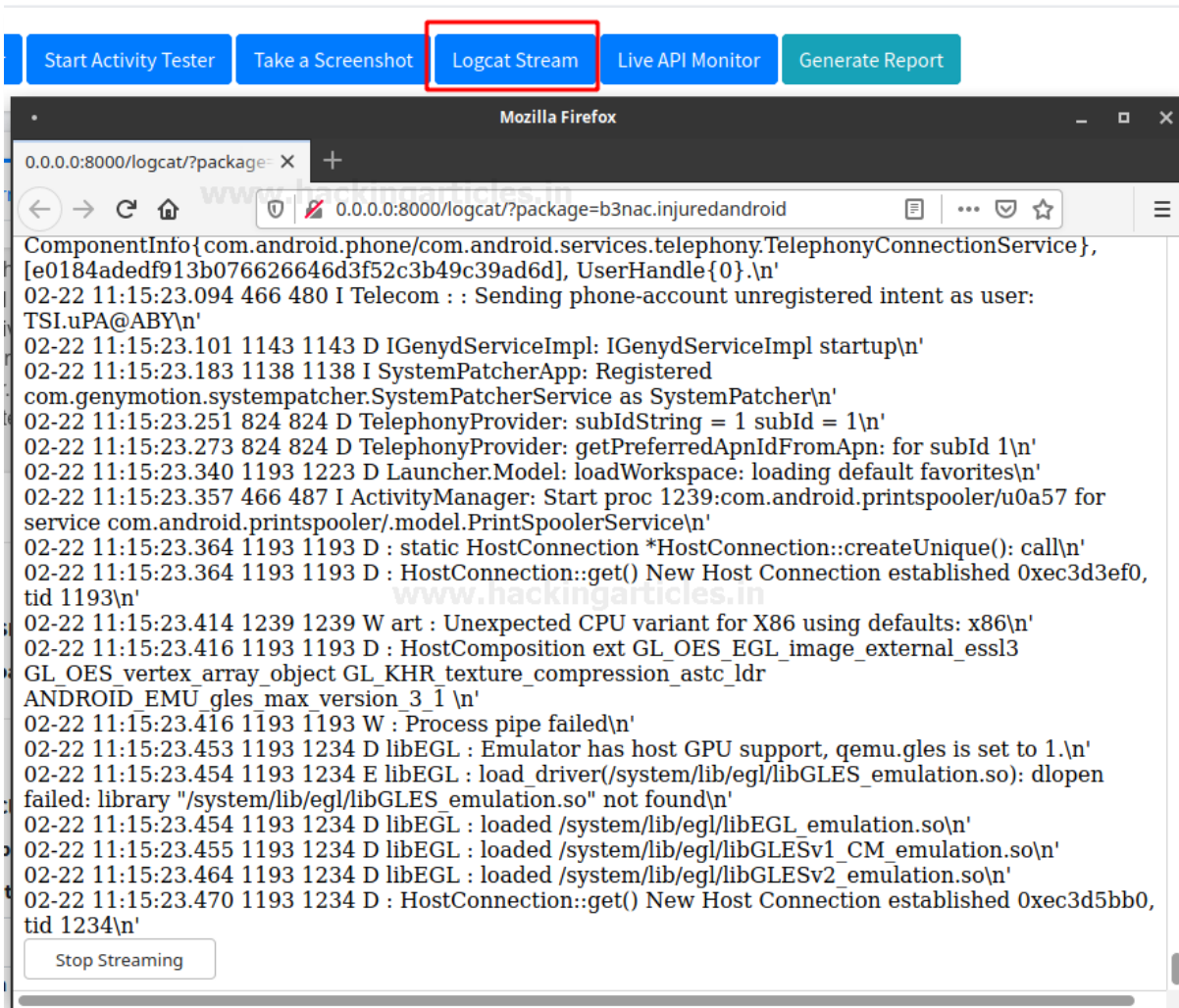
```
1 // https://codeshare.frida.re/@dzonerzy/aesinfo/
2
3 Java.perform(function () {
4     var complete_bytes = new Array();
5     var index = 0;
6     var secretKeySpecDef =
7     Java.use('javax.crypto.spec.SecretKeySpec');
8     var ivParameterSpecDef =
9     Java.use('javax.crypto.spec.IvParameterSpec');
10    var cipherDef = Java.use('javax.crypto.Cipher');
11    var cipherDoFinal_1 = cipherDef.doFinal.overload();
12    var cipherDoFinal_2 = cipherDef.doFinal.overload('[B');
13    var cipherDoFinal_3 = cipherDef.doFinal.overload('[B',
14    'int');
15    var cipherDoFinal_4 = cipherDef.doFinal.overload('[B',
16    'int', 'int');
17    var cipherDoFinal_5 = cipherDef.doFinal.overload('[B',
18    'int', 'int', '[B');
19    var cipherDoFinal_6 = cipherDef.doFinal.overload('[B',
20    'int', 'int', '[B', 'int');
21    var cipherUpdate_1 = cipherDef.update.overload('[B');
22    var cipherUpdate_2 = cipherDef.update.overload('[B', 'int',
23    'int');
24    var cipherUpdate_3 = cipherDef.update.overload('[B', 'int',
25    'int', '[B');
26    var cipherUpdate_4 = cipherDef.update.overload('[B', 'int',
27    'int', '[B', 'int');
28    var secretKeySpecDef_init_1 =
29    secretKeySpecDef.$init.overload('[B', 'java.lang.String');
30    var secretKeySpecDef_init_2 =
31    secretKeySpecDef.$init.overload('[B', 'int', 'int',
```

Available Scripts (Use CTRL to choose multiple) [Load](#)

default
helper
jni_hook_by_address
aes_key

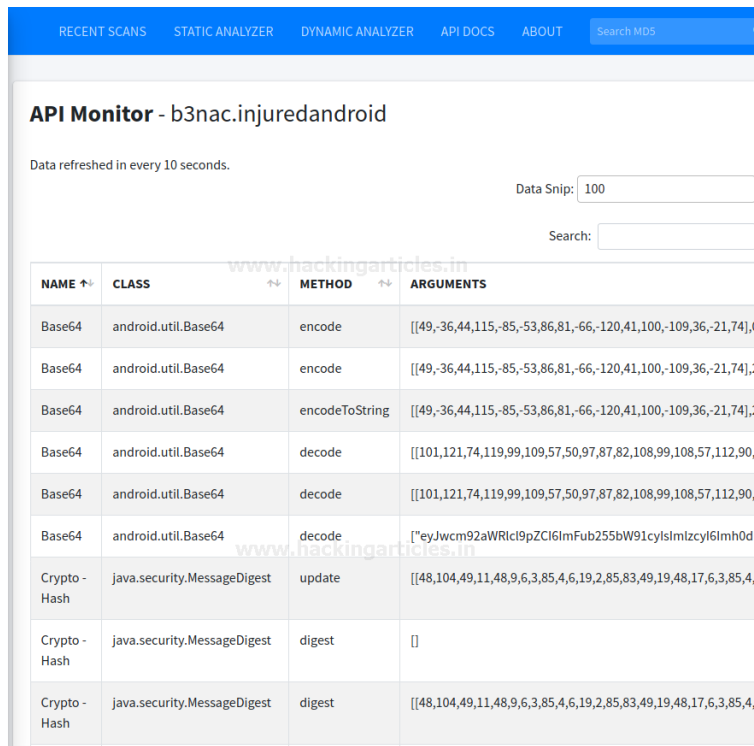
Logcat Stream

Logcat can also be viewed in MobSF's native environment. There's a button at the top menu that lets one view this.



API Monitor

Just like logcat monitors device logs, APIs can also be monitored. APKs use various APIs in real-time to perform various functions, for example, the Base64 library.



The screenshot shows the API Monitor web application interface. At the top, there is a navigation bar with links: RECENT SCANS, STATIC ANALYZER, DYNAMIC ANALYZER, API DOCS, and ABOUT. A search bar labeled 'Search MD5' is also present. The main heading is 'API Monitor - b3nac.injuredandroid'. Below this, it states 'Data refreshed in every 10 seconds.' There are two input fields: 'Data Snip:' with the value '100' and 'Search:'. The main content is a table with the following columns: NAME, CLASS, METHOD, and ARGUMENTS. The table lists several API calls, including Base64 encode/decode and Crypto-Hash update/digest methods.

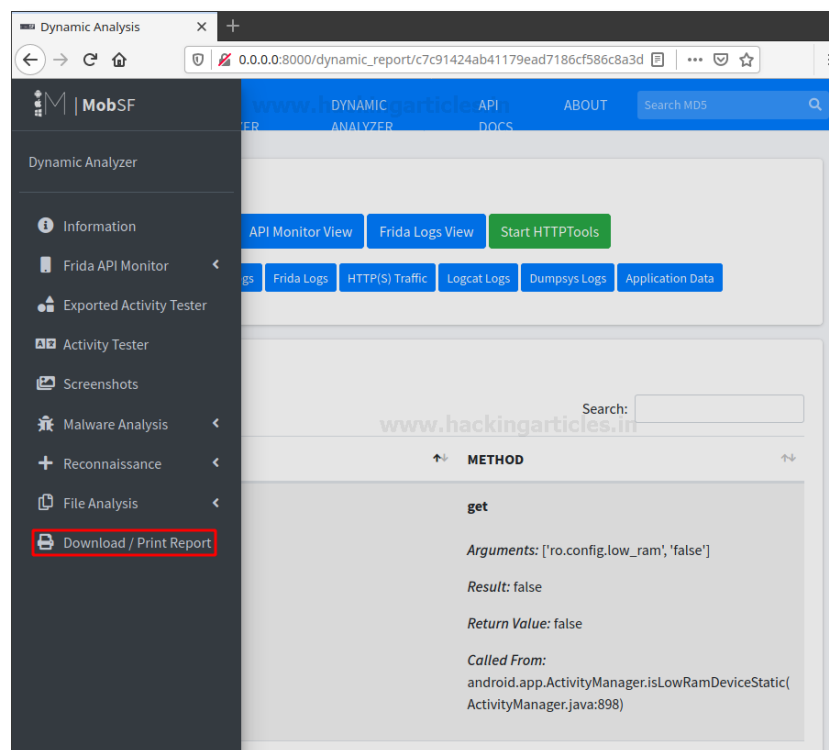
NAME	CLASS	METHOD	ARGUMENTS
Base64	android.util.Base64	encode	[[49,-36,44,115,-85,-53,86,81,-66,-120,41,100,-109,36,-21,74],...
Base64	android.util.Base64	encode	[[49,-36,44,115,-85,-53,86,81,-66,-120,41,100,-109,36,-21,74],...
Base64	android.util.Base64	encodeToString	[[49,-36,44,115,-85,-53,86,81,-66,-120,41,100,-109,36,-21,74],...
Base64	android.util.Base64	decode	[[101,121,74,119,99,109,57,50,97,87,82,108,99,108,57,112,90,...
Base64	android.util.Base64	decode	[[101,121,74,119,99,109,57,50,97,87,82,108,99,108,57,112,90,...
Base64	android.util.Base64	decode	["eyJwcm92aWRlcl9pZCI6ImFub255bW91cytsImlzcyl6Imh0dl",...
Crypto - Hash	java.security.MessageDigest	update	[[48,104,49,11,48,9,6,3,85,4,6,19,2,85,83,49,19,48,17,6,3,85,4,...
Crypto - Hash	java.security.MessageDigest	digest	[]
Crypto - Hash	java.security.MessageDigest	digest	[[48,104,49,11,48,9,6,3,85,4,6,19,2,85,83,49,19,48,17,6,3,85,4,...

Hence, if a function is using this API and decrypting a value we can see that value here and decode that. For example, down below you can see the return value of once such function in Base64.

RESULT	RETURN VALUE
"[object Object]"	"77100119115995411876861087143105671081"
"[object Object]"	"77100119115995411876861087143105671081"
"MdWSC6vLVIG+iClkkyTrSg=="	"MdWSC6vLVIG+iClkkyTrSg=="
"[object Object]"	"12334112114111118105100101114951051003"
"[object Object]"	"12334112114111118105100101114951051003"
"[object Object]"	"12334112114111118105100101114951051003"

Downloading Reports

Once you have done the analysis, it is possible to download the report by sliding the menu bar slider on the left-hand side and click generate the report.

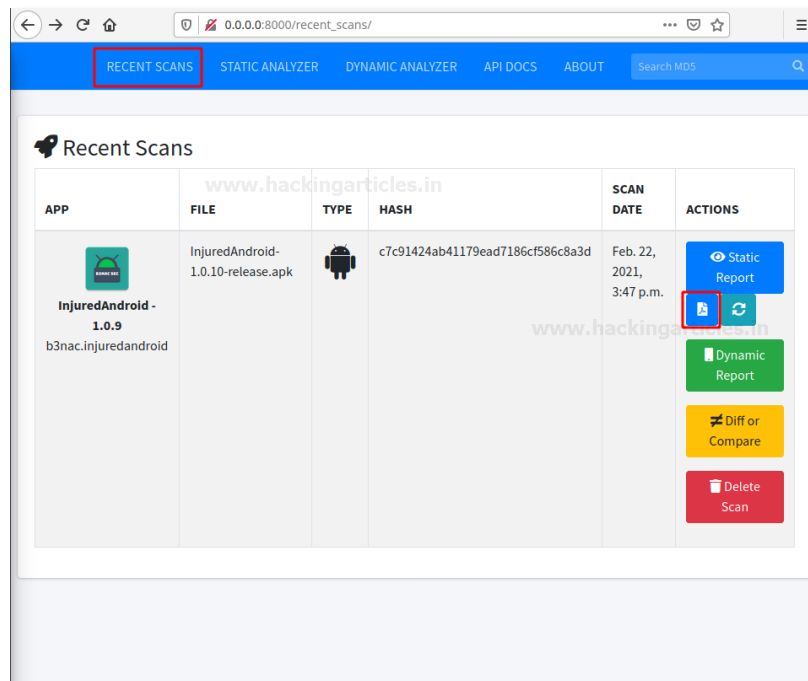


You might notice some errors while generating reports. To resolve this, you can follow the below command and install **wkhtmltopdf** module:

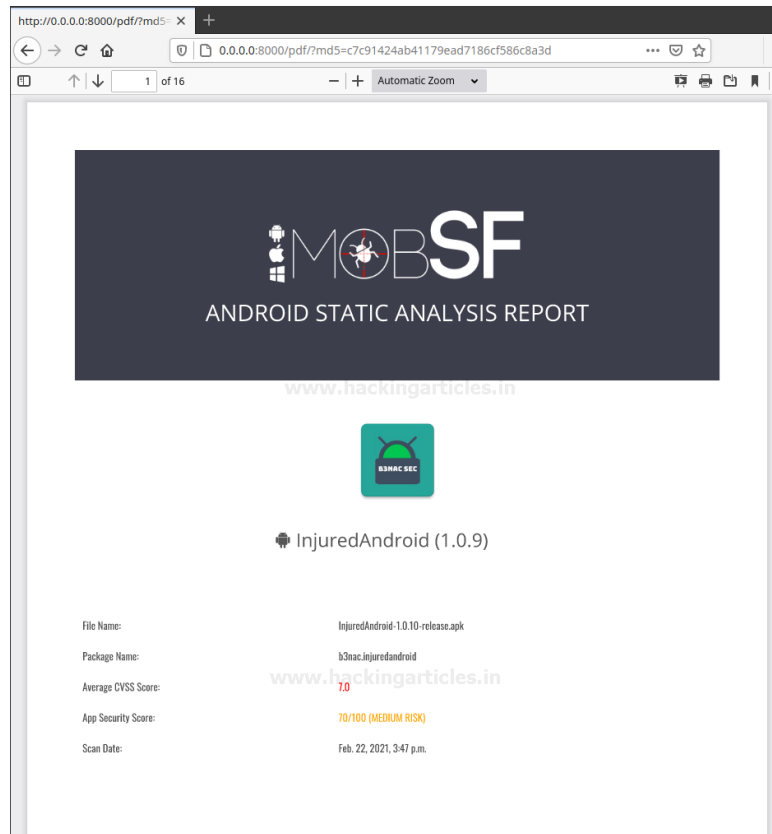
```
apt-get install wkhtmltopdf
```

```
root@hex:/home/hex# apt-get install wkhtmltopdf
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following NEW packages will be installed:
  wkhtmltopdf
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 203 kB of archives.
After this operation, 1,111 kB of additional disk space will be used.
Get:1 http://in.archive.ubuntu.com/ubuntu focal/universe amd64 wkhtmltopdf amd64 0.12.5-1build1 [203 kB]
Fetched 203 kB in 1s (232 kB/s)
Selecting previously unselected package wkhtmltopdf.
(Reading database ... 462931 files and directories currently installed.)
Preparing to unpack .../wkhtmltopdf_0.12.5-1build1_amd64.deb ...
Unpacking wkhtmltopdf (0.12.5-1build1) ...
Setting up wkhtmltopdf (0.12.5-1build1) ...
Processing triggers for man-db (2.9.1-1) ...
root@hex:/home/hex#
```

Now, once again if you click on a recent scan bar, you'll see static and dynamic report generation options.



The report looks something like this:



JOIN OUR TRAINING PROGRAMS

